

Pyrrmit: hosting the local corpus, embeddings and app

Report of October 9, 2026. Covers where to host the app now that retrieval runs on the local corpus (`RETRIEVAL_BACKEND=local` , `CHAT_MODE=agent`), what it costs, the embedding model choice, and a CPU (ONNX int8) benchmark. Sizes and timings were measured on the development machine (Intel i7-12700H, 14 cores, RTX 4060 Laptop 8 GB, 16 GB RAM). Prices come mostly from third-party trackers as of September–October 2026 and should be checked on the official pages before committing.

Summary

- **Embedding model:** Snowflake `snowflake-arctic-embed-1-v2.0` (568M parameters, 1024 dims), run locally. It is **not on OpenRouter**. Its official **ONNX int8** version runs on CPU at **34 ms per query**, uses **0.9 GB RAM** and keeps the same recall as the GPU model. No GPU is needed to serve the app, and there is no need to change models.
 - **Recall:** R@5 0.944 / R@10 0.956 on the 45-question eval. Going from 5 to 10 chunks recovers one passage; the remaining misses are query or label issues, not a k problem.
 - **Data to host:** corpus store 1.5 GB (read-only), PDFs 18.4 GB, users and chats 73 MB (already on Neon).
 - **Recommended now: one VPS** (4 vCPU / 8 GB / 80 GB disk, about €7–15 per month) holding the backend, the store and the PDFs on disk. Neon free tier keeps users and chats, and Vercel or the same VPS serves the frontend. No code change is needed: the ONNX int8 CPU runtime is now in the app.
 - **Later, at scale:** containers plus **Neon pgvector** for the corpus (about 2.5–3 GB, about \$1 per month of storage) and **Cloudflare R2** for the PDFs (about \$0.13 per month). This needs the store and agent tools ported to Postgres, and PDFs read from object storage.
 - **The real cost is the LLM:** an agent answer costs about **\$0.007–0.011** (GPT-6 Luna via OpenRouter, benchmark data), so about **\$10 per 1,000 questions**. That exceeds hosting beyond a few hundred questions a month.
 - **Rebuilding everything** (crawl, extraction, embeddings, store) takes **about 5 h on the GPU machine**, or **about 1 day with OCR**. It must stay on a GPU: CPU indexing would take about 53 h.
-

1. What has to be hosted

Component	Size / need	Where it lives today
Next.js frontend	Light	frontend/ , local
FastAPI backend	Long-running process. Agent answers take 30–110 s and keep running in a background thread when the tab closes. The embedding model and the store stay loaded.	local, port 8000
Corpus store (data/index/store/st-arctic-1-v2/)	corpus.sqlite 750 MB (chunks, FTS5 BM25, places), vectors.npy 714 MB (348,736 × 1024 float16), read-only	local disk
Plan registry + gazetteer	plans.jsonl 8.7 MB, municipalities.json 0.25 MB	local disk
Crawled PDFs + OCR text	raw/web/*/pdf 18.4 GB (7,798 files), raw/web/*/ocr 8 MB	local disk
Users, chat sessions, messages	73 MB	Neon (Postgres)
Pipeline (crawl, OCR, embeddings)	Needs the GPU	development PC

Serverless functions (Vercel, Lambda) are a poor fit for the backend. The answers are long, the generation outlives the HTTP connection, and the model plus the store take several seconds to load.

2. Embedding model

2.1 Current model

Model	Snowflake/snowflake-arctic-embed-1-v2.0 (default LOCAL_RAG_MODEL=st-arctic-1-v2 , localrag/service.py)
Parameters	about 568M (XLM-RoBERTa large base, the same family as bge-m3)
Weights	about 1.1 GB in fp16, 2.2 GB in fp32 (safetensors download: 2.3 GB)
Output	1024 dims, CLS pooling + normalisation, 8k-token context, multilingual, Apache-2.0
Query prefix	query:

2.2 How it was chosen

The candidates were multilingual models under 1B parameters with good MTEB multilingual / Scandinavian retrieval scores that fit the 8 GB laptop GPU (localrag/models.toml). API models were added as references. All of them were run on the 45 hand-labelled use-case questions (localrag/questions.jsonl), with the kommun filter active and the question translated to Swedish, which is the dominant factor (bge-m3: 0.63 with the English query, 0.91 with the Swedish one).

Model (Swedish query)	Where	R@5	R@1	OpenRouter price (\$/M tokens)
arctic-embed-l-v2	local	0.92	0.82	not available
gemini-embedding-001	API	0.93	0.66	0.15
multilingual-e5-large	API	0.92	0.71	0.01
bge-m3	local / API	0.91	0.78	0.01
Qwen3-Embedding-8B	API	0.91	0.71	0.01
Qwen3-Embedding-4B	API	0.90	–	0.02
voyage-4	API	0.90	–	0.06
multilingual-e5-large-instruct	local	0.89	–	–
Qwen3-Embedding-0.6B	local	0.81	–	–
KBLab sentence-bert (Swedish only)	local	0.70	0.51	–
BM25 alone (Swedish query)	–	0.76	0.51	–

arctic matched the best API models at R@5 and had the best R@1 and MRR, for free, on the local GPU. BM25 fusion and the bge-reranker-v2-m3 cross-encoder did not improve on dense retrieval (–4 to –8 points).

The MTEB Scandinavian leaderboard (formerly SEB) now lives inside the main MTEB leaderboard. No reliable, current Swedish-retrieval ranking was found online. The in-house eval on the real legal corpus remains the better judge.

2.3 Recall at 5 vs 10

The current eval (corpus re-chunked on October 8) gives **R@1 0.833, R@5 0.944, R@10 0.956, MRR 0.869**. The five passages missed at 5 sit at ranks 10, 12 and 15, and two are not in the top 20.

- Passing 10 chunks instead of 5 recovers **one** passage. The misses come from the query wording or the labels, not from k.
- The agent's `search` tool already returns 8 results per pool (up to 15) and reformulates its queries, so single-query recall underestimates what the agent finds.
- With 45 questions, one passage is worth about 1.7 points. **Differences under 3 points between models are noise.**

2.4 OpenRouter

arctic-embed-l-v2 is **not listed** on OpenRouter (checked on the public `/api/v1/embeddings/models` endpoint). It is only sold through Snowflake Cortex, at about \$0.07–0.10 per M tokens according to trackers.

Embedding models on OpenRouter not yet tested on this eval:

Model	\$/M tokens
perplexity/pplx-embed-v1-4b	0.03
perplexity/pplx-embed-v1-0.6b	0.004
voyageai/voyage-4-large	0.12
voyageai/voyage-4-lite	0.02
google/gemini-embedding-2	0.20
openai/text-embedding-3-large	0.13
nvidia/nemotron-3-embed-1b	free
liquid/lfm-2.5-embedding-350m	free

- **Testing them all** on the eval sub-corpus (18,802 chunks, 16.8 M characters, about 5 M tokens) would cost **about \$2.5**.
- **Switching the app to an API model** means re-indexing the whole corpus (about 90 M tokens): about \$1 with bge-m3 or Qwen3-8B, about \$5 with voyage-4, about \$13.5 with gemini-embedding-001. Query embeddings would then cost almost nothing (about 30 tokens each).
- **Query and index must use the same model**. Given the ONNX results below, switching is not necessary.

3. Running the model without a GPU (ONNX int8)

Snowflake publishes official ONNX exports in the model repository (`onnx/model1.onnx` fp32, `onnx/model1_int8.onnx` 570 MB, plus fp16 and q4). The test kept the existing index (built in fp16 on the GPU) and changed only the query encoder. Recall was measured on the 45 questions in dense mode with the Swedish query. Throughput was measured on 128 real chunks (about 1,500 characters each).

Encoder	Load	Model RAM	Query (median)	Indexing	R@1 / R@5 / R@10 / MRR
torch GPU fp16 (current app)	41 s	1.7 GB	23 ms	93 chunks/s	0.833 / 0.944 / 0.956 / 0.869
torch CPU fp32	39 s	3.4 GB	141 ms	1.8 chunks/s	0.822 / 0.944 / 0.956 / 0.865
ONNX CPU fp32	10 s	1.5 GB	133 ms	1.2 chunks/s	0.822 / 0.944 / 0.956 / 0.865
ONNX CPU int8	4 s	0.9 GB	34 ms	1.9 chunks/s	0.811 / 0.944 / 0.956 / 0.857
ONNX CPU int8, 4 threads	3 s	0.9 GB	43 ms	1.0 chunks/s	same
torch CPU fp32, 4 threads	31 s	3.4 GB	117 ms	1.0 chunks/s	same as torch CPU

- **Vector fidelity**: compared with torch fp32, the cosine is 1.0000 for ONNX fp32 and GPU fp16, and **0.98** for int8 (min 0.975 on queries, 0.967 on documents). R@5 and R@10 do not move; one passage drops from rank 1 to 2.
- **Search in the disk store on CPU** (348k chunks, no GPU): law pool (3,651 rows) 18 ms; one kommun 1–24 ms; unfiltered local pool (no kommun detected) **0.8 s**.
- **Conclusion**: ONNX int8 is the way to serve the app without a GPU. It is 4× faster than torch on CPU, uses 4× less RAM and loses no measurable recall, and it is compatible with the existing index. **Indexing must stay on the GPU**: at 1.8 chunks/s, re-embedding the full corpus on CPU would take about 53 h.
- **Now in the app**: `EMBEDDING_RUNTIME=auto` (default, `backend/embeddings.py`) runs torch fp16 on a CUDA GPU, and otherwise the ONNX int8 export on the CPU (`onnx_file` in `localrag/models.toml`). `onnxruntime` is in the `.[local]` extras. Through the app, the eval gives R@5 0.944 in both modes.

4. Databases

4.1 What is on Neon today

Table	Rows	Size	Content
law_chunks	2,244	32 MB	vector(3072) (OpenAI text-embedding-3-large), about 590 characters per chunk
document_chunks	2,375	33 MB	only 7 documents
chat_messages, chat_sessions, users, accounts, sessions	about 370	about 1 MB	app data
Total		73 MB	

- The database runs PostgreSQL 17 with **pgvector 0.8.0**, which supports `halfvec`, HNSW / IVFFlat and iterative index scans (filtered ANN).
- **There is no vector index:** every search is an exact scan, which is fine at this size. HNSW on `vector` is limited to 2,000 dims, so the 3,072-dim column could not be indexed anyway.
- Installable extensions: `pg_search` 0.15 (ParadeDB, real BM25 based on Tantivy), `pg_trgm`, `unaccent`. The native `tsvector` also has a `swedish` configuration (Snowball stemmer).
- The existing embeddings are incompatible with arctic (different model and 3,072 dims), so moving the corpus there means new tables.

4.2 Footprint of the local corpus in Postgres (estimate)

Item	Measured locally	In Postgres
Chunk text	314 M characters (about 900 per chunk)	about 0.32 GB
Metadata (title, url, header, ids...)	about 104 M characters	about 0.12 GB
Vectors, <code>halfvec(1024)</code>	714 MB	about 0.72 GB (1.43 GB as <code>vector float32</code>)
Full-text index (<code>tsvector</code> + GIN, or <code>pg_search</code>)	FTS5: 90 MB	about 0.2–0.5 GB
B-tree indexes (kommun, doc_id, chunk_id)		about 0.05 GB
<code>places</code> (770k rows) + plan registry		about 0.07 GB
Total without HNSW		about 1.5–2 GB
Optional HNSW index on <code>halfvec</code>		+0.8–1 GB, about 2.5–3 GB in total

At about \$0.35 per GB-month (Neon Launch), that is **\$0.6–1 per month** of storage. It does not fit the free plan (0.5 GB per project). The initial bulk load inflates restore history for a while, and building HNSW needs RAM (scale compute up to 2–4 CU during the build). Most searches are filtered (law pool, or one kommun of a few thousand chunks), so an exact scan with a B-tree on `kommun_code` is fast without HNSW. HNSW only matters for unfiltered searches.

4.3 Vector database options

HNSW is an approximate nearest-neighbour index. It becomes necessary around millions of vectors or at high query rates. At 350k vectors, mostly searched with a filter, an exact search is fast and has perfect recall. **Supabase and Neon both run pgvector, which has HNSW:** dedicated vector databases do not have a better algorithm.

Their advantages are filter-aware graph traversal, quantisation, sparse vectors for hybrid search, and scale (millions to billions of vectors, high QPS).

Option	Model	Free tier	Paid entry	Notes
Neon + pgvector	Serverless Postgres, scales to zero	0.5 GB / project, 100 CU-h	Launch: \$0.106 / CU-h + \$0.35 / GB-month, no base fee	Already used for app data: one database for everything, SQL filters, Swedish <code>tsvector</code> . Cold start after suspension.
Supabase	Postgres + pgvector (same engine), always-on instance	500 MB, paused after 1 week idle	Pro \$25 / month (8 GB)	Also bundles auth and file storage. Nothing extra on the vector side.
Qdrant Cloud	Dedicated vector DB (Rust, HNSW with payload filtering)	1 GB RAM / 4 GB disk, suspended after 1 week idle	Resource-based (calculator)	int8-quantised vectors (about 350 MB) would fit the free cluster. Text and metadata would still live elsewhere.
Pinecone	Serverless, proprietary	2 GB, 1M reads / 2M writes per month	Builder \$20 / month; Standard \$50 minimum	Storage about \$0.33 / GB-month.
Turbopuffer	Built on object storage	none	Launch minimum \$16–64 / month (sources disagree)	Very cheap at large scale.
LanceDB	Embedded, files on S3 / R2	open source	–	Closest to the current file-based design (memmap).

5. Object storage for the PDFs

Object storage keeps whole files ("objects") addressed by a key (for example `raw/web/0115/pdf/<id>.pdf`) and serves them over HTTP. It is not a database (no queries) and not a disk (no partial writes, though ranged reads work). The pattern is: the database holds text, vectors and the file key; the bucket holds the file.

Service	Storage	Egress	Free tier	18.4 GB of PDFs
Cloudflare R2	\$0.015 / GB-month	free	10 GB, 1M class A + 10M class B operations / month	about \$0.13 / month
AWS S3 Standard	\$0.023 / GB-month	\$0.09 / GB beyond 100 GB / month	–	about \$0.42 / month + egress

R2 is the default choice. S3 only makes sense if the backend runs on AWS, because S3 → EC2 traffic in the same region is free. Today the backend reads PDFs from the local disk (`routers/docs.py`, and the agent tools `view_map_area` / `view_pdf_page` in `agentic/tools.py`); moving them to a bucket needs a small read-through cache.

6. Rebuilding the pipeline

Estimated from the last run (crawl manifests, file timestamps) and the measurements above, on the GPU machine.

Step	Command	Time	Basis
Kommuner (Wikidata)	<code>corpus.municipalities</code>	about 1 min	
Laws (Riksdagen, 15 SFS)	<code>corpus.laws</code>	about 1 min	
Crawl of 286 kommun sites	<code>corpus.crawl</code>	about 3–4 h	32.4 kommun-hours at concurrency 12; the longest kommun took 35 min; 82k requests, 18.7 GB of PDFs
OCR of scanned PDFs (optional)	<code>corpus.ocr</code>	about 18 h	777 files, EasyOCR on the GPU
Text extraction + chunking	<code>corpus.extract</code> , <code>corpus.chunk</code>	about 5–10 min	
Embeddings (348k chunks)	<code>localrag.index</code>	about 1–1.5 h	93 chunks/s measured in fp16 on the GPU
Disk store + plan registry	<code>localrag.store</code> , <code>corpus.plans</code>	about 3 min	
Model download	automatic	about 5–15 min	2.3 GB (or 570 MB for ONNX int8)

Total: about 5 h without OCR, about 1 day with OCR, plus about 15 min to install CUDA torch on a fresh machine.

- The crawl is slow on purpose (1 request per second per host). A faster machine does not help; the biggest sites set the pace.
- Incremental runs are much faster. Finished kommuner are skipped (`--force` to recrawl), and indexing only embeds new or changed chunks, because chunk ids include a text hash.
- Without a GPU, this is impractical: about 53 h for the embeddings and days of OCR.
- A full GPU rebuild could also run on a rented GPU by the hour, but the development PC is enough.

7. Hosting options and costs

7.1 Option 1: one server (recommended now)

One VPS, for example at Hetzner (Germany or Finland: EU and close to Sweden), with 4 vCPU, 8 GB RAM and 80 GB disk. The backend, the SQLite store and the PDFs are on its disk; ONNX int8 serves the queries. Users and chats stay on Neon (free plan). The frontend runs on Vercel or on the same server behind Caddy (HTTPS).

Item	Monthly cost
VPS 4 vCPU / 8 GB / 80 GB	about €7–15 (CX32 listed at €6.80 in a 2026 comparison; Hetzner has raised prices, so check)
Neon (users, chats)	€0 (free plan)
Vercel (frontend)	€0 Hobby (\$20 Pro for commercial use)
Total hosting	about €7–15

- **Pros:** no code change (the app already reads local files and falls back to ONNX int8 on the CPU). Data updates are an `rsync` of the store and the new PDFs after a rebuild on the PC.
- **Cons:** server administration (HTTPS, system updates), and a single instance.

7.2 Option 2: managed services (when there are real users)

The backend runs in a container (Fly.io, Railway, Render, Cloud Run with always-allocated CPU), the corpus moves to Neon pgvector, and the PDFs go to R2.

Item	Monthly cost
Container with 4 GB RAM, always on	about \$20–40 (no reliable 2026 list price found; check each provider's calculator)
Neon Launch (about 3 GB corpus)	about \$1 of storage + a few dollars of compute (scales to zero)
R2 (18.4 GB, 10 GB free)	about \$0.13
Total	about \$25–45

- **Pros:** stateless containers, horizontal scaling, no server to administer.
- **Work needed:** port `localrag/store.py` and `_DiskCorpus` (`agentic/tools.py`) to Postgres (`halfvec`, B-tree filters, `tsvector('swedish')` or `pg_search`), and read the PDFs from R2 with a local cache.

7.3 LLM cost (dominant)

Benchmark run (agent, medium effort)	Answers	Mean cost per answer	Mean time per answer
Feasibility cases	28	\$0.0071	31 s
Case-law cases	100	\$0.0086	42–44 s
Plankarta cases	40	\$0.0110	107 s

Usage	LLM cost per month
1,000 questions	about \$10
10,000 questions	about \$100

Query embeddings are local, so they cost nothing. Beyond a few hundred questions a month, the LLM costs more than all the hosting combined.

7.4 Recommendation

1. Done: the ONNX int8 CPU runtime is in the app (`EMBEDDING_RUNTIME=auto`).
2. Deploy option 1: one VPS + Neon (free) + Vercel. Expected cost **about €10 per month + LLM**.
3. Keep the pipeline on the development PC (GPU) and push the store and new PDFs with `rsync`.
4. Optional: evaluate the untested OpenRouter embedding models (about \$2.5). Only worth it if a model is clearly better than arctic.
5. Move to option 2 (Neon pgvector + R2 + containers) when traffic or operations justify it.

Caveats

- Hosting and storage prices come mostly from third-party trackers (September–October 2026), except Cloudflare R2 (official, January 2026). Check the official pages before committing.
- The recall figures come from 45 questions; differences under about 3 points are not significant.
- The CPU timings come from a 14-core laptop CPU; a 4-thread run is given as a proxy for a smaller server.

Sources

1. OpenRouter, embedding models API: <https://openrouter.ai/api/v1/embeddings/models>
2. Snowflake arctic-embed-l-v2.0, ONNX files: <https://huggingface.co/Snowflake/snowflake-arctic-embed-l-v2.0/tree/main/onnx>
3. Superlinked, arctic-embed-l-v2.0 specs: <https://superlinked.com/models/snowflake-snowflake-arctic-embed-l-v2-0>
4. Cloudprice, arctic-embed-2-l (Snowflake Cortex price): <https://cloudprice.net/models/snowflake-arctic-embed-2-l>
5. FutureAGI, Snowflake Cortex price calculator: <https://futureagi.com/llm-cost-calculator/snowflake-cortex/snowflake-arctic-embed-l-v2-0>
6. Scandinavian Embedding Benchmark (moved into MTEB): <https://github.com/KennethEnevoldsen/Scandinavian-Embedding-Benchmark/blob/main/docs/index.md>
7. MMTEB paper: <https://arxiv.org/html/2502.13595v3>
8. MTEB leaderboard: <https://huggingface.co/spaces/mteb/leaderboard>
9. NVIDIA Llama-nemotron-embed-1b-v2 model card: <https://build.nvidia.com/nvidia/llama-nemotron-embed-1b-v2/modelcard>
10. Jetadmin, Neon review 2026 (pricing): <https://www.jetadmin.io/blog/neon-review/>
11. Makerkit, Neon pricing calculator: <https://makerkit.dev/pricing-calculator/neon>
12. Prisma, Prisma Postgres vs Neon pricing 2026: <https://www.prisma.io/blog/prisma-postgres-vs-neon-pricing-2026>
13. Costbench, Qdrant free plan 2026: <https://www.costbench.com/software/vector-databases/qdrant/free-plan/>
14. Toolradar, Turbopuffer pricing: <https://toolradar.com/tools/turbopuffer/pricing>
15. Usagepricing, Turbopuffer price change (July 2026): <https://usagepricing.com/blueprint/activity/turbopuffer-2026-07-14-price-change>
16. Pinecone, manage cost: <https://docs.pinecone.io/guides/manage-cost>
17. Pinecone pricing guide 2026: <https://apicalculators.com/blog/pinecone-pricing-guide-2026>
18. DEV, Pinecone vs Weaviate vs Qdrant (2026): <https://dev.to/devtoolpicks/pinecone-vs-weaviate-vs-qdrant-for-indie-hackers-in-2026-real-costs-honest-verdict-11do>
19. Cloudflare R2 pricing: <https://developers.cloudflare.com/r2/pricing>
20. Cloudflare R2 calculator: <https://r2-calculator.cloudflare.com/>
21. DEV, AWS S3 pricing for beginners: <https://dev.to/sh20raj/aws-s3-pricing-for-beginners-3fc2>
22. AWS Data Transfer Hub, cost example: <https://docs.aws.amazon.com/solutions/latest/data-transfer-hub/cost.html>
23. DEV, cheapest cloud servers comparison: <https://dev.to/stargazermedia/what-are-cheapest-cloud-servers-out-there-in-2025-comparison-377a>
24. Hetzner price change summary: <https://news.hada.io/topic?id=18010>
25. DEV, Railway vs Render vs Heroku: https://dev.to/alex_aslam/deploy-nodejs-apps-like-a-boss-railway-vs-render-vs-heroku-zero-server-stress-5p3
26. DEV, Fly.io alternatives: <https://dev.to/code42cate/5-awesome-flyio-alternatives-3cea>
27. In-house measurements: `backend/data/eval/20261007-100634.json`, `backend/data/agentic/bench-*agent-medium-explore-medium*.json`, the retrieval comparison in `reports/pyrmit_technical_report.pdf`, and the ONNX benchmark of October 9, 2026.